

마이크로프로세서설계실험

텀 프로젝트 보고서

12조

고경빈, 우희연

I. 조원소개 및 조원별 기여 내용

고경빈: gpio 인터럽트, 암호 소스코드 작성, 장치 아이디어 제시, 보고서 블록다이어그램, 동작설명 작성

우희연: LCD 소스코드 작성, 보고서 LCD 동작 설명, 결과 작성

II. 수행 내용

GPIO 인터럽트, 7-SEGMENT, 키패드, LCD를 사용한 암호 통신장치를 제작했다.

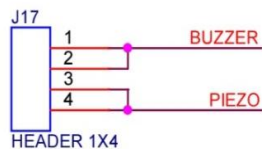
1. 장치를 키면 LCD 화면에 “Push the button”이 표시된다.
2. 잘못된 버튼을 누르면 피에조가 작동하고, 올바른 버튼을 누르면 LCD 에 통신암호가 표기되는 동시에 키패드와 7-SEG 와 LED 가 작동된다.
3. 통신암호는 stdio.h 의 rand()함수를 통해 난수를 발생시켜서 만들었고, sprintf 함수를 통해 문자열로 변환해서 lcd 에 표시했다.
4. 통신암호에서 특정 자리수 4 자리(7 번째→1 번째→3 번째→10 번째)를 순서대로 입력하면, 정답일 경우 변수 asum 에 저장된다.
5. 7-SEG 에 입력한 값이 동시에 표시될 것이다.
6. 사용자가 키패드로 정답을 입력하면 통신 받은 메시지가 표시되면서 LED 가 켜진다.

III. 입출력 할당 테이블

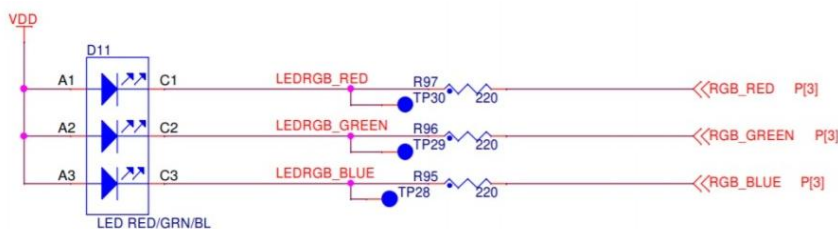
- Buzzer 연결: D2

결선 방법

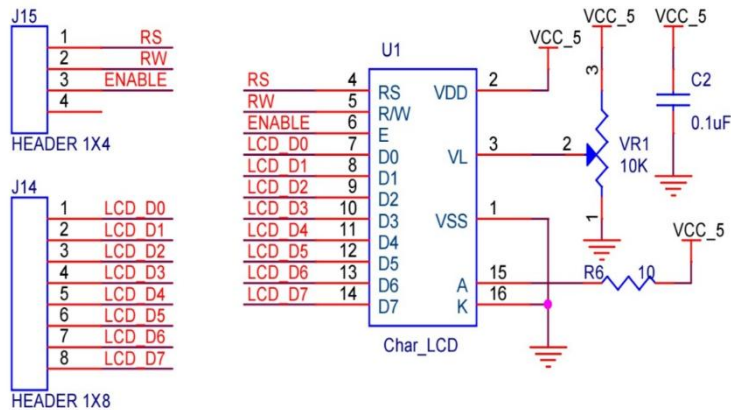
J17-1: PTD2 (LS1-1(BUZZER))



- LED 연결: D0(Blue), D15(Red), D16(Green)



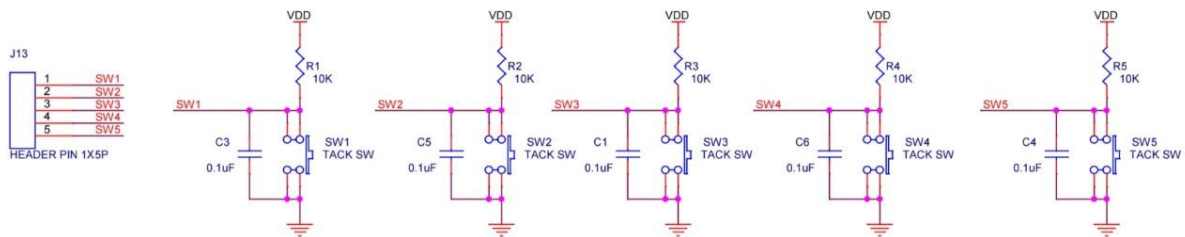
- LCD 디스플레이 연결: D8~D14



결선 방법

- J14-5: PTD8 (LCD-11(D4))
- J14-6: PTD9 (LCD-12(D5))
- J14-7: PTD10 (LCD-11(D6))
- J14-8: PTD11 (LCD-12(D7))
- J15-3: PTD12 (LCD-6(ENABLE))
- J15-2: PTD13 (LCD-5(RW))
- J15-1: PTD14 (LCD-4(RS))

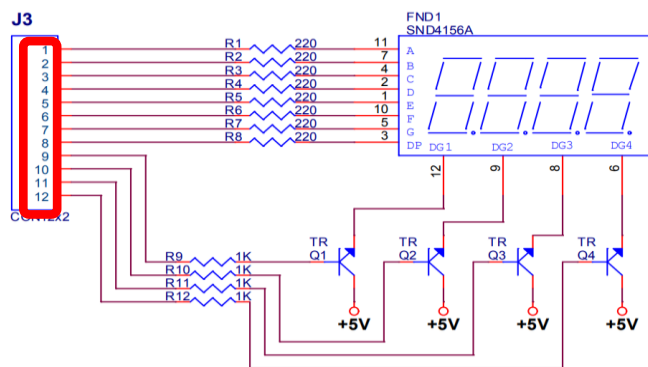
- Switch 결선: A10~A15



결선 방법

- J13-1: PTA11 (SW1)
- J13-2: PTA12 (SW2)
- J13-3: PTA13 (SW3)
- J13-4: PTA14 (SW4)
- J13-5: PTA15 (SW5)

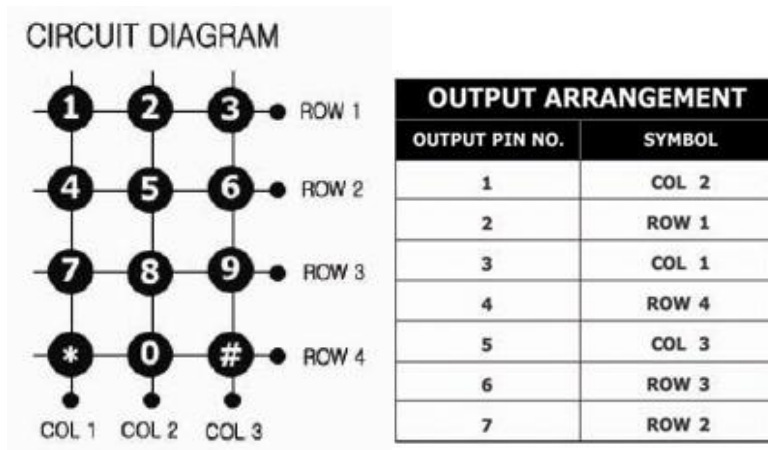
- 7-Segment display 연결: C1~C3, C7~C13, C15



결선 방법

- J3-1: PTC1 (FND-11(A))
 - J3-2: PTC2 (FND-7(B))
 - J3-3: PTC3 (FND-4(C))
 - J3-4: PTC12 (FND-2(D))
 - J3-5: PTC13 (FND-1(E))
 - J3-6: PTC15 (FND-10(F))
 - J3-7: PTC7 (FND-5(G))
 - J3-8: PTC8 (FND-12(DG1))
 - J3-9: PTC9 (FND-9(DG2))
 - J3-10: PTC10 (FND-8(DG3))
 - J3-11: PTC11 (FND-6(DG4))
- (C4,5,6 오작동으로 인해 변경)

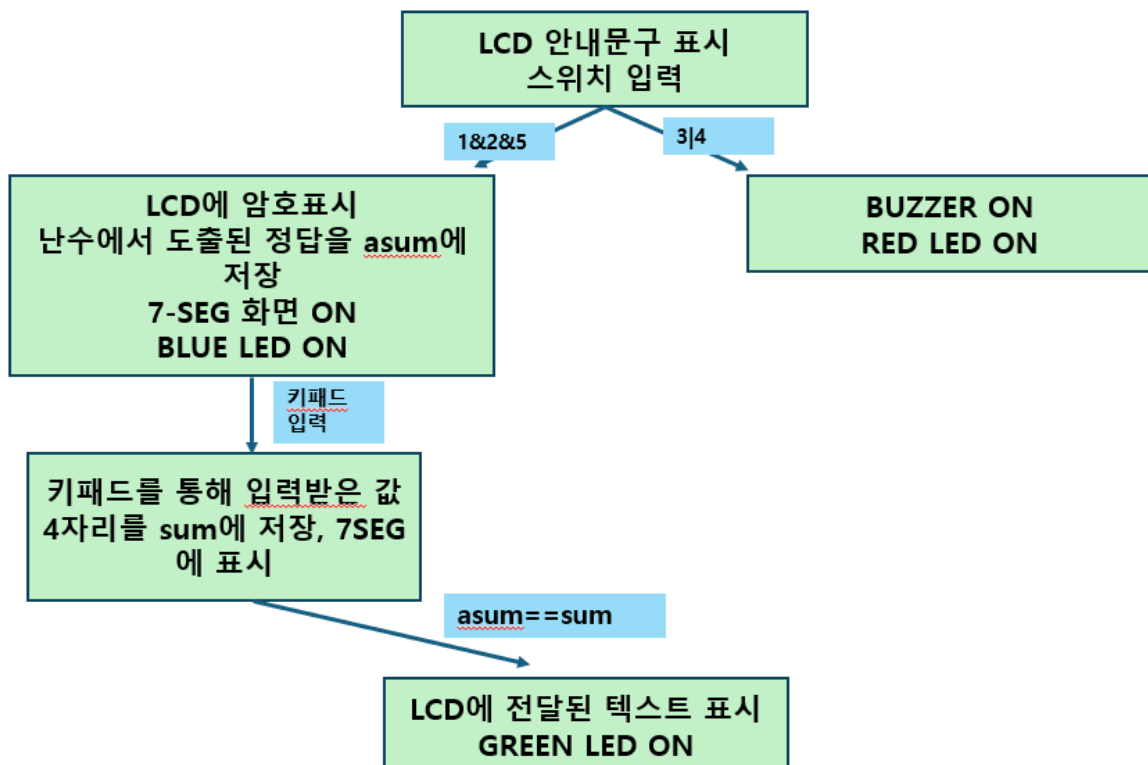
- Keypad 연결: E0~E3, E12, E14, E15



결선 방법

- PTE12 (Keypad-1st column[3] / Output)
- PTE14 (Keypad-2nd column[1] / Output)
- PTE15 (Keypad-3rd column[5] / Output)
- PTE0 (Keypad-1st row[2] / Input)
- PTE1 (Keypad-2nd row[7] / Input)
- PTE2 (Keypad-3rd row[6] / Input)
- PTE3 (Keypad-4th row[4] / Input)

IV. 블록 다이어그램



V. 실행 코드 동작 설명

1. 장치 실행시 rand() 함수로 난수를 생성하고 이를 k에 대입한다. sprintf 함수를 통해 문자열로 변환해 msg[50]에 저장한다. 또한 난수의 특정 자리수를 추출해서(a1: 7 번째 자리 수, a2: 3 번째 자리 수, a3: 8 번째 자리 수, a4: 10 번째 자리 수) 정답을 만들고 asum 변수에 저장한다.
2. 가능한 시간의 한 tick 마다 데이터 비트 D0~D7를 전송한다. 그러나, Busy flag가 1인 동안 LCD 모듈에 데이터가 수신되지 않기 때문에 충분한 delay를 설정하는 것으로 안내문자를 포함한 모든 데이터가 LCD에 수신되게 한다. text_lcd 함수를 통해 안내문자("push the button")를 출력한다.
3. gpio 인터럽트를 통해 SW1~5이 눌리면 k1~k5가 1이되게 한다, SW1,2,5를 각각 한 번씩 누르면 다음단계로 넘어가고, SW3 또는 SW4를 누르면 RED LED와 Buzzer가 울리게 한다.
4. 올바른 스위치를 눌렀을 때 text_lcd 함수를 통해 난수가 표시되는데, 이때 난수를 출력하기 전에 lcd_init과 i=0을 설정하는 것으로 이전에 출력된 데이터를 초기화 시켜서 bit가 겹치는 것을 방지한다. 사용자는 이를 보고 송신자와 수신자 사이 약속한 자리수를 추출해서 정답을 입력한다. 숫자입력은 keypad를 통해 입력받는데, 동일 숫자도 입력되게 하기 위해 2초마다 LPIT0 CH1안의 코드 key=keyscan()가 실행되게 해서 키값을 받았다. 이러면 main의 반복문 안에서 key값이 10보다 작다면 sum에 더해지게 되고 key값은 100으로 초기화 되서 sum 반복문이 멈추게 된다. 따라서 입력할 숫자를 2초씩 누르고 있으면 원하는 값 4자리를 입력할 수 있다.
5. 정답인 asum과 키패드 입력값 sum이 일치하게 되면 LCD에 수신문자(여기선 correct)가 표시되게 되고 GREEN LED가 ON되므로써 수신이 정상적으로 되었음을 알려준다.

VI. LCD 설명

- LCD 동작 코드(lcd1602A.c): LCD 모듈을 4비트 모드로 제어하기 위한 초기화 및 데이터 전송 절차를 구현한 것이다.
 1. delay_100ns 함수: LPIT를 사용해서 정확한 시간 지연을 생성한다. 타이머 채널 0의 값을 설정하고 완료될 때까지 기다린다.
 2. lcdEN 및 lcdNEN 함수: LCD 제어 핀 중 하나인 EN을 Set(High)와 Clear(low)로 설정한다.
 3. lcdinit 함수: LCD의 주요 명령어를 설정하는 것으로 LCD 컨트롤러를 초기화한다.
 4. lcdinput 함수: LCD 명령어를 상위 4비트와 하위 4비트로 나누어서 전송한다.

5. Lcdcharinput 함수: LCD 에 문자 데이터를 전송한 뒤에, RS 핀이 데이터 모드로 설정된다.
- LCD 헤더 파일 (lcd1602.h):
1. lcdinit 함수: LCD 모듈 초기화 시에, LCD 의 동작 모드와 화면 클리어, 커서 설정을 차례대로 수행한다.
2. lcdinput 함수: LCD 에 상위 4 비트와 하위 4 비트로 나뉘서 총 16 비트의 명령어 데이터를 전송한다.
3. lcdcharinput 함수: RS 핀을 설정해서 LCD 가 데이터 모드임을 알린 후, 출력을 위한 1 바이트의 문자를 송신한다.
4. busycheck 함수: LCD 가 현재 명령어나 데이터를 처리 중이라면 Busy 상태이기 때문에, LCD 가 다음 명령을 받을 준비가 되어있는지 확인한다.

VII. 결과

장치를 실행하자마자, LCD화면에 버튼을 누르라는 안내문이 표시된 것을 확인할 수 있다. 먼저, 스위치 3번이나 4번을 눌러보는 것으로 잘못된 스위치를 눌렀을 때 오류를 나타내는 Buzzer 소리와 Red LED가 정상 작동하는 것을 확인했다. 장치를 재실행 하고나서, 스위치 1, 2, 5번을 순서대로 누르면, 옳은 스위치를 눌렀다는 의미를 나타내는 Blue LED가 켜지면서 LCD에 난수가 표시된 것을 확인할 수 있다. Keypad에 아무 숫자나 입력하다가 난수의 8번째, 4번째, 10번째, 1번째 자리에 해당하는 숫자를 차례대로 누르면 green LED가 켜지면서 LCD에 정답을 의미하는 안내문이 표시된 것을 확인할 수 있다.

VIII. 고찰

이번 프로젝트를 통해 실습시간 때 배웠던 여러 GPIO의 사용과 인터럽트, 타임 인터럽트를 토대로 응용해봤고, LCD의 제어법을 배워서 같이 적용해봤다. 여러 기능을 연계해서 적용해봄으로써 임베디드 설계 방식을 잘 배워 볼 수 있었다. 추후 개선점이라면은 키패드를 누르고 2초 뒤 떼더라도 인터럽트 발동 주기마다 눌렀던 키가 반복해서 입력되는데 , 떼면 키가 입력이 안되도록 개선해 볼 수 있을 것 같다. 실질적인 통신이 되려면 문구랑 추출 자릿수가 바뀔 수 있어야 되는데 마지막 시간에 배웠던 UART통신을 더 배워서 구현해 보고 싶다.

1. LCD 동작코드 (lcd1602A.c)
2. LCD 헤더파일 (lcd1602A.h)
3. 실행 코드

```

#include "device_registers.h"           // Device header
#include "clocks_and_modes.h"
#include "lcd1602A.h"
#include "stdlib.h"
#include "stdio.h"
int lpit0_ch0_flag_counter = 0; /*< LPIT0 timeout counter */
unsigned int External_PIN=0; /* External_PIN:SW External input Assignment */
unsigned int num,num0,num1,num2,num3 =0;
unsigned int k1,k2,k3,k4,k5 =0;
unsigned int a1,a2,a3,a4;
unsigned int i=0;
unsigned int Delaytime = 0; /* Delay Time Setting Variable*/
int key=0;
int sum=0;
int prekey=50;
unsigned int FND_SEL[4]={0x0100,0x0200,0x0400,0x0800};
unsigned int j=0; /*FND select pin index */
unsigned int Dtime = 0; /* Delay Time Setting Variable*/

void WDOG_disable (void)
{
    WDOG->CNT=0xD928C520;    /* Unlock watchdog      */
    WDOG->TOVAL=0x0000FFFF;  /* Maximum timeout value    */
    WDOG->CS = 0x00002100;    /* Disable watchdog        */
}

void PORT_init (void)
{

```

```

    PTD->PDDR |= 1<<9 | 1<<10 | 1<<11 | 1<<12 | 1<<13 | 1<<14 | 1<<8;
    PCC->PCCn[PCC_PORTD_INDEX] &= ~PCC_PCCn_CGC_MASK;
    PCC->PCCn[PCC_PORTD_INDEX] |= PCC_PCCn_PCS(0x001);

```

```

PCC->PCCn[PCC_PORTD_INDEX] |= PCC_PCCn_CGC_MASK;
PCC->PCCn[PCC_FTM2_INDEX]  &= ~PCC_PCCn_CGC_MASK;
PCC->PCCn[PCC_FTM2_INDEX]  |= (PCC_PCCn_PCS(1)|
PCC_PCCn_CGC_MASK);      //Clock = 80MHz

//Pin mux
PORTD->PCR[9]= PORT_PCR_MUX(1);
PORTD->PCR[10]= PORT_PCR_MUX(1);
PORTD->PCR[11]= PORT_PCR_MUX(1);
PORTD->PCR[12]= PORT_PCR_MUX(1);
PORTD->PCR[13]= PORT_PCR_MUX(1);
PORTD->PCR[14]= PORT_PCR_MUX(1);
PORTD->PCR[8]= PORT_PCR_MUX(1);

//LED
PORTD->PCR[0]= PORT_PCR_MUX(1);
PORTD->PCR[15]= PORT_PCR_MUX(1);
PORTD->PCR[16]= PORT_PCR_MUX(1);
PTD->PDDR |= 1<<0 | 1<<15 | 1<<16;

//BUZZER
PORTD->PCR[1]= PORT_PCR_MUX(1);
PTD->PDDR |= (1<<1);
PORTD->PCR[2]= PORT_PCR_MUX(1);
PTD->PDDR |= (1<<2);

//A
PCC->PCCn[PCC_PORTA_INDEX]|=PCC_PCCn_CGC_MASK;

PORTA->PCR[11] |= PORT_PCR_MUX(1); // Port A11 mux = GPIO
PORTA->PCR[11] |=(10<<16); // Port A11 IRQC : interrupt on Falling-
edge
PORTA->PCR[12] |= PORT_PCR_MUX(1); // Port A12 mux = GPIO
PORTA->PCR[12] |=(10<<16); // Port A12 IRQC : interrupt on Falling-
edge
PORTA->PCR[13] |= PORT_PCR_MUX(1); // Port A13 mux = GPIO

```



```

PORTA->PCR[13] |= (10<<16); // Port A12 IRQC : interrupt on Falling-
edge
PORTA->PCR[14] |= PORT_PCR_MUX(1); // Port A13 mux = GPIO
PORTA->PCR[14] |= (10<<16); // Port A12 IRQC : interrupt on Falling-
edge
PORTA->PCR[15] |= PORT_PCR_MUX(1); // Port A11 mux = GPIO
PORTA->PCR[15] |= (10<<16); // Port A11 IRQC : interrupt on Falling-
edge
PTA->PDDR &= ~((1<<11)|(1<<12)|(1<<13)|(1<<14)|(1<<15));

/* Enable clock for PORT E */
PCC-> PCCn[PCC_PORTE_INDEX] = PCC_PCCn_CGC_MASK;

PTE->PDDR |= 1<<12|1<<14|1<<15;          /* Port E12,E14-E15:
Data Direction = output */
PTE->PDDR &= ~(1<<0);    /* Port E0: Data Direction= input (default)
*/
PTE->PDDR &= ~(1<<1);    /* Port E1: Data Direction= input (default)
*/
PTE->PDDR &= ~(1<<2);    /* Port E2: Data Direction= input (default)
*/
PTE->PDDR &= ~(1<<3);    /* Port E3: Data Direction= input (default)
*/

PORTE->PCR[0] =
PORT_PCR_MUX(1)|PORT_PCR_PFE(1)|PORT_PCR_PE(1)|PORT_PCR_PS(0); /* Port E0:
MUX = GPIO, input filter enabled, internal pull-down enabled */
PORTE->PCR[1] =
PORT_PCR_MUX(1)|PORT_PCR_PFE(1)|PORT_PCR_PE(1)|PORT_PCR_PS(0); /* Port E1:
MUX = GPIO, input filter enabled, internal pull-down enabled */
PORTE->PCR[2] =
PORT_PCR_MUX(1)|PORT_PCR_PFE(1)|PORT_PCR_PE(1)|PORT_PCR_PS(0); /* Port E2:
MUX = GPIO, input filter enabled, internal pull-down enabled */

```

```

        PORTE->PCR[3] =
PORT_PCR_MUX(1)|PORT_PCR_PFE(1)|PORT_PCR_PE(1)|PORT_PCR_PS(0); /* Port E3:
MUX = GPIO, input filter enabled, internal pull-down enabled */
PORTE->PCR[0] |= (9 << 16);
PORTE->PCR[1] |= (9 << 16);
PORTE->PCR[2] |= (9 << 16);
PORTE->PCR[3] |= (9 << 16);

```

```

        PORTE->PCR[12] = PORT_PCR_MUX(1); /* Port E12: MUX = GPIO
*/
        PORTE->PCR[14] = PORT_PCR_MUX(1); /* Port E14: MUX = GPIO
*/
        PORTE->PCR[15] = PORT_PCR_MUX(1); /* Port E15: MUX = GPIO
*/

```

```

        PCC->PCCn[PCC_PORTC_INDEX] = PCC_PCCn_CGC_MASK; /* Enable
clock for PORT C */

```

```

        PTC->PDDR |= (1 << 1);
        PTC->PDDR |= (1 << 2);
        PTC->PDDR |= (1 << 3);
        PTC->PDDR |= (1 << 15);
        PTC->PDDR |= (1 << 12);
        PTC->PDDR |= (1 << 13);
        PTC->PDDR |= (1 << 7);
        PTC->PDDR |= 1 << 8 | 1 << 9 | 1 << 10 | 1 << 11;

```

```

        PORTC->PCR[1] = PORT_PCR_MUX(1);
        PORTC->PCR[2] = PORT_PCR_MUX(1);
        PORTC->PCR[3] = PORT_PCR_MUX(1);
        PORTC->PCR[15] = PORT_PCR_MUX(1);
        PORTC->PCR[13] = PORT_PCR_MUX(1);
        PORTC->PCR[7] = PORT_PCR_MUX(1);

```

```

        PORTC->PCR[8] = PORT_PCR_MUX(1);
        PORTC->PCR[9] = PORT_PCR_MUX(1);
        PORTC->PCR[10] = PORT_PCR_MUX(1);
        PORTC->PCR[11] = PORT_PCR_MUX(1);
        PORTC->PCR[12] = PORT_PCR_MUX(1);

    }

void LPIT0_init (uint32_t delay)
{
    uint32_t timeout;

    /*!
     * LPIT Clocking:
     * =====
     */
    PCC->PCCn[PCC_LPIT_INDEX] = PCC_PCCn_PCS(6);    /* Clock Src = 6
(SPLL2_DIV2_CLK)*/
    PCC->PCCn[PCC_LPIT_INDEX] |= PCC_PCCn_CGC_MASK; /* Enable clk
to LPIT0 regs */

    /*!
     * LPIT Initialization:
     */
    LPIT0->MCR |= LPIT_MCR_M_CEN_MASK; /* DBG_EN=0: Timer chans
stop in Debug mode */

                                     /* DOZE_EN=0: Timer chans
are stopped in DOZE mode */

                                     /* SW_RST=0: SW reset does
not reset timer chans, regs */

```

```

/* M_CEN=1: enable module
clk (allows writing other LPIT0 regs) */

timeout=delay* 40;
LPIT0->TMR[0].TVAL = timeout;      /* Chan 0 Timeout period: 40M clocks */
LPIT0->TMR[0].TCTRL |= LPIT_TMR_TCTRL_T_EN_MASK;
/* T_EN=1: Timer channel is enabled */
/* CHAIN=0: channel chaining is disabled */
/* MODE=0: 32 periodic counter mode */
/* TSOT=0: Timer decrements immediately
based on restart */
/* TSOI=0: Timer does not stop after timeout */
/* TROT=0 Timer will not reload on trigger */
/* TRG_SRC=0: External trigger source */
/* TRG_SEL=0: Timer chan 0 trigger source is
selected*/

//keyscan

LPIT0->MIER = 0x02;

LPIT0->TMR[1].TVAL = 80000000;      /* Chan 0 Timeout period: 40M
clocks */
LPIT0->TMR[1].TCTRL |= LPIT_TMR_TCTRL_T_EN_MASK;
}

void delay_us (volatile int us){
    LPIT0_init(us);      /* Initialize PIT0 for 1 second timeout */
    while (0 == (LPIT0->MSR & LPIT_MSR_TIF0_MASK)) {} /* Wait for LPIT0 CH0
Flag */
    lpit0_ch0_flag_counter++;      /* Increment LPIT0 timeout
counter */
    LPIT0->MSR |= LPIT_MSR_TIF0_MASK; /* Clear LPIT0 timer flag 0
*/
}

```

```

}

void NVIC_init_IRQs(void){
    S32_NVIC->ICPR[1] |= 1 << (59%32); // Clear any pending IRQ59
    S32_NVIC->ISER[1] |= 1 << (59%32); // Enable IRQ59
    S32_NVIC->IP[59] = 0x0; //Priority 11 of 15
    //keyscan
    S32_NVIC->ICPR[1] |= 1 << (49 % 32);
    S32_NVIC->ISER[1] |= 1 << (49 % 32);
    S32_NVIC->IP[49] = 0x0B;

}

```

```

void PORTA_IRQHandler(void){

    //PORTA_Interrupt State Flag Register Read
    if((PORTA->ISFR & (1 << 11)) != 0){
        External_PIN=1;
    }
    else if((PORTA->ISFR & (1 << 12)) != 0){
        External_PIN=2;
    }
    else if((PORTA->ISFR & (1 << 13)) != 0){
        External_PIN=3;
    }

    else if((PORTA->ISFR & (1 << 14)) != 0){
        External_PIN=4;
    }
    else if((PORTA->ISFR & (1 << 15)) != 0){
        External_PIN=5;
    }
}

```

```

// External input Check Behavior Assignment
switch (External_PIN){

case 1: k1=1;
        External_PIN=0;
        break;

case 2:
        k2 = 1;
        External_PIN=0;
        break;

case 3:
        k3 = 1;
        External_PIN=0;
        break;

case 4:
        k4 = 1;
        External_PIN=0;
        break;

case 5:
        k5 = 1;
        External_PIN=0;
        break;

default:
        break;

}

PORTA->PCR[11] |= 0x01000000; // Port Control Register ISF bit '1' set
PORTA->PCR[12] |= 0x01000000; // Port Control Register ISF bit '1' set
PORTA->PCR[13] |= 0x01000000; // Port Control Register ISF bit '1' set
PORTA->PCR[14] |= 0x01000000; // Port Control Register ISF bit '1' set
PORTA->PCR[15] |= 0x01000000; // Port Control Register ISF bit '1' set
}

```

```

void text_lcd(char *mess){
    while(mess[i] != '\0'){
        if(i%16 == 0){
            lcdinput(0x80+0x40);
            if(i%32==0){
                lcdinit();
            }
            delay_us(500);
        }
        lcdcharinput(mess[i]); // 1(first) row text-char send to LCD module
        delay_us(200);
        i++;
    }
}

```

```

void seg (int nom){
    switch(nom){
        case 0:
            PTC-> PSOR |= 1<<1; //
            PTC-> PSOR |= 1<<2; //
            PTC-> PSOR |= 1<<3; //
            PTC-> PSOR |= 1<<12; //
            PTC-> PSOR |= 1<<13; //
            PTC-> PSOR |= 1<<15; //
            PTC-> PCOR |= 1<<7; //
            break;
        case 1:
            PTC-> PCOR |= 1<<1;
            PTC-> PSOR |= 1<<2;

```

```

PTC-> PSOR |= 1<<3;
PTC-> PCOR |= 1<<12;
PTC-> PCOR |= 1<<13;
PTC-> PCOR |= 1<<15;
PTC-> PCOR |= 1<<7;
break;
case 2:
PTC-> PSOR |= 1<<1;//PTD1; // FND A ON
PTC-> PSOR |= 1<<2;//PTD2; // FND B ON
PTC-> PCOR |= 1<<3;//PTD3; //
PTC-> PSOR |= 1<<12;//PTD4; // FND D ON
PTC-> PSOR |= 1<<13;//PTD5; // FND E ON
PTC-> PCOR |= 1<<15;//PTD6; //
PTC-> PSOR |= 1<<7;//PTD7; // FND G ON
break;
case 3:
PTC-> PSOR |= 1<<1;//PTD1; // FND A ON
PTC-> PSOR |= 1<<2;//PTD2; // FND B ON
PTC-> PSOR |= 1<<3;//PTD3; // FND C ON
PTC-> PSOR |= 1<<12;//PTD4; // FND D ON
PTC-> PCOR |= 1<<13;//PTD5; //
PTC-> PCOR |= 1<<15;//PTD6; //
PTC-> PSOR |= 1<<7;//PTD7; // FND G ON
break;
case 4:
PTC-> PCOR |= 1<<1;//PTD1; //
PTC-> PSOR |= 1<<2;//PTD2; // FND B ON
PTC-> PSOR |= 1<<3;//PTD3; // FND C ON
PTC-> PCOR |= 1<<12;//PTD4; //
PTC-> PCOR |= 1<<13;//PTD5; //
PTC-> PSOR |= 1<<15;//PTD6; // FND F ON
PTC-> PSOR |= 1<<7;//PTD7; // FND G ON
break;
case 5:
PTC-> PSOR |= 1<<1;//PTD1; // FND A ON

```



```

PTC-> PCOR |= 1<<2;//PTD2; //
PTC-> PSOR |= 1<<3;//PTD3; // FND C ON
PTC-> PSOR |= 1<<12;//PTD4; // FND D ON
PTC-> PCOR |= 1<<13;//PTD5; //
PTC-> PSOR |= 1<<15;//PTD6; // FND F ON
PTC-> PSOR |= 1<<7;//PTD7; // FND G ON
break;
case 6:
PTC-> PSOR |= 1<<1;//PTD1; // FND A ON
PTC-> PCOR |= 1<<2;//PTD2; //
PTC-> PSOR |= 1<<3;//PTD3; // FND C ON
PTC-> PSOR |= 1<<12;//PTD4; // FND D ON
PTC-> PSOR |= 1<<13;//PTD5; // FND E ON
PTC-> PSOR |= 1<<15;//PTD6; // FND F ON
PTC-> PSOR |= 1<<7;//PTD7; // FND G ON
break;
case 7:
PTC-> PSOR |= 1<<1;//PTD1; // FND A ON
PTC-> PSOR |= 1<<2;//PTD2; // FND B ON
PTC-> PSOR |= 1<<3;//PTD3; // FND C ON
PTC-> PCOR |= 1<<12;//PTD4; //
PTC-> PCOR |= 1<<13;//PTD5; //
PTC-> PSOR |= 1<<15;//PTD6; // FND F ON
PTC-> PCOR |= 1<<7;//PTD7; //
break;
case 8:
PTC-> PSOR |= 1<<1;//PTD1; // FND A ON
PTC-> PSOR |= 1<<2;//PTD2; // FND B ON
PTC-> PSOR |= 1<<3;//PTD3; // FND C ON
PTC-> PSOR |= 1<<12;//PTD4; // FND D ON
PTC-> PSOR |= 1<<13;//PTD5; // FND E ON
PTC-> PSOR |= 1<<15;//PTD6; // FND F ON
PTC-> PSOR |= 1<<7;//PTD7; // FND G ON
break;
case 9:

```

```

        PTC-> PSOR |= 1<<1;//PTD1; // FND A ON
        PTC-> PSOR |= 1<<2;//PTD2; // FND B ON
        PTC-> PSOR |= 1<<3;//PTD3; // FND C ON
        PTC-> PSOR |= 1<<12;//PTD4; // FND D ON
        PTC-> PCOR |= 1<<13;//PTD5; //
        PTC-> PSOR |= 1<<15;//PTD6; // FND F ON
        PTC-> PSOR |= 1<<7;//PTD7; // FND G ON
        break;

    }

}

void Seg_out(int number)
{
    Dtime = 1000;

    num3=(number/1000)%10;
    num2=(number/100)%10;
    num1=(number/10)%10;
    num0= number%10;

    // 1000??? ??
    PTC->PSOR = FND_SEL[j];
    PTC->PCOR =0x7f;
    seg(num3);
    delay_us(Dtime);
    PTC->PCOR = 0xff;
    j++;

    // 100??? ??
    PTC->PSOR = FND_SEL[j];
    PTC->PCOR =0x7f;
    seg(num2);

```

```

delay_us(Dtime);
    PTC->PCOR = 0xffff;
    j++;

    // 10??? ??
    PTC->PSOR = FND_SEL[j];
    PTC->PCOR = 0x7f;
    seg(num1);
delay_us(Dtime);
PTC->PCOR = 0xffff;
    j++;

    // 1??? ??
    PTC->PSOR = FND_SEL[j];
    PTC->PCOR = 0x7f;
    seg(num0);
delay_us(Dtime);
    PTC->PCOR = 0xffff;
    j=0;
}

int Kbuff = 0;

int KeyScan(void)
{

    Dtime = 1000;

    PTE->PSOR |= 1 < 12;
    delay_us(Dtime);
    if(PTE->PDIR &(1 < 0))Kbuff=1;    //1
    if(PTE->PDIR &(1 < 1))Kbuff=4;    //4
    if(PTE->PDIR &(1 < 2))Kbuff=7;    //7
    if(PTE->PDIR &(1 < 3))Kbuff=11;   //*

```

```

PTE->PCOR |=1<<12;

PTE->PSOR |=1<<14;
delay_us(Dtime);
if(PTE->PDIR & (1<<0))Kbuff=2;      //2
if(PTE->PDIR & (1<<1))Kbuff=5;      //5
if(PTE->PDIR & (1<<2))Kbuff=8;      //8
if(PTE->PDIR & (1<<3))Kbuff=0;      //0
PTE->PCOR |=1<<14;

PTE->PSOR |=1<<15;
delay_us(Dtime);
if(PTE->PDIR & (1<<0))Kbuff=3;      //3
if(PTE->PDIR & (1<<1))Kbuff=6;      //6
if(PTE->PDIR & (1<<2))Kbuff=9;      //9
if(PTE->PDIR & (1<<3))Kbuff=12;     //#
PTE->PCOR |=1<<15;

return Kbuff;
}

void LPIT0_Ch1_IRQHandler (void)
{
    key=KeyScan();

    LPIT0->MSR |= LPIT_MSR_TIF0_MASK; /* Clear LPIT0 timer flag 0 */
}

int main(void)
{
    WDOG_disable();
    PORT_init();          /* Configure ports */
    SOSC_init_8MHz();      /* Initialize system oscillator for 8 MHz xtal */
    SPLL_init_160MHz();    /* Initialize SPLL to 160 MHz with 8 MHz SOSC */

```

```

        NormalRUNmode_80MHz(); /* Init clocks: 80 MHz sysclk & core,
40 MHz bus, 20 MHz flash */
        NVIC_init_IRQs();      /* Enable desired interrupts and priorities */
        LPIT0_init(10);
int asum=0;
        num=0;
        unsigned long long k;
        k=rand();
        char msg[50];

        PTD->PSOR|=(1<<0)|(1<<15)|(1<<16);
        PTD->PCOR |= (1<<2);
        sprintf(msg,"%llu",k);

        text_lcd("push the button");

        a1=(k/1000000)%10;
        a2=(k/100)%10;
        a3=(k/1)%10;
        a4=(k/1000000000)%10;
        asum=a1*1000+a2*100+a3*10+a4;
        asum%=10000;

while(1) {
        if(k1&k2&k5){
        PTD->PCOR|=(1<<0);
        lcdinit();
        i=0;

        text_lcd(msg);
        while(1){

```

```

                                if(key<10 )
                                {
sum=10*sum+key;
sum%=10000;

key=100;

}

                                Seg_out(sum);
if(sum==asum) {
    lcdinit();
    i=0;
    text_lcd("correct");
PTD->PCOR|=(1<<16);
PTD->PSOR|=(1<<0);

k1=0;
    break;
}

                                }

                                }

else{

                                if(k3|k4) {
PTD->PCOR|=(1<<15);
PTD->PSOR|=(1<<2);

                                }

}

```

}

}